

Stage 5: Optional Extensions

Heuristic Improvements, Application-Level Simulation, and Future Work

Team Members: Name 1, Name 2, Name 3

CS240: Algorithm Design and Analysis, Spring 2026

1 Purpose of the Extension Stage

The required project already implements and evaluates a classical graph partitioning method. This optional stage describes extensions that go beyond the minimum requirements. The goal is not to claim state-of-the-art graph partitioning performance, but to show how the core algorithmic idea can be adapted, improved, and connected more deeply to the modern application domain of distributed optimization.

The project includes one implemented extension and several natural future extensions. The implemented extension is a lightweight consensus-averaging simulation that connects graph partitioning to cross-worker communication cost. The future extensions include weighted communication costs, stronger preprocessing, sparse spectral methods, multilevel partitioning, and more realistic distributed optimization simulations.

2 Implemented Extension: Consensus Communication Simulation

The main report evaluates graph partitioning not only by pure graph metrics but also by a small distributed consensus simulation. Each vertex holds a scalar value, and neighboring vertices repeatedly exchange information. The update resembles standard average consensus:

$$x_i^{(t+1)} = x_i^{(t)} + \alpha \sum_{j:(i,j) \in E} \left(x_j^{(t)} - x_i^{(t)} \right),$$

where α is chosen based on the maximum degree. The target value is the initial average of all node values.

The partition is used to count how many messages cross worker boundaries. For each edge whose endpoints are assigned to different partitions, the simulation counts two directed cross-worker messages per iteration. This produces a communication metric that is directly related to the cut:

$$\text{cross messages} \approx 2T \cdot |\{(u, v) \in E : p(u) \neq p(v)\}|,$$

where T is the number of consensus iterations.

This extension gives the graph partitioning objective an application-level interpretation. Lower cut weight means fewer cross-worker messages, which is exactly the kind of cost that distributed systems try to reduce.

3 Observed Extension Results

The average cross-worker message counts are:

$$\text{greedy} = 15460, \quad \text{spectral} = 11360, \quad \text{Kernighan-Lin} = 10440.$$

These results mirror the cut-weight comparison. Kernighan-Lin refinement has the lowest average cut and therefore the lowest average cross-worker traffic. This supports the idea that a classical graph partitioning algorithm can be used as a communication-placement tool in distributed optimization.

The consensus error itself is nearly identical across partitions on the same graph. This is expected because the simulation keeps the original graph topology and averaging dynamics unchanged. The partition affects only where messages are placed, not which vertices communicate. This result is useful because it clarifies what the current extension measures: communication placement cost rather than convergence acceleration.

4 Possible Extension 1: Weighted Communication Costs

The current experiments use mostly unweighted or uniformly weighted graphs. A more realistic distributed optimization system may have nonuniform communication costs. For example, some dependencies may require large tensor transfers, some worker pairs may communicate through slower network links, and some variables may be more strongly coupled than others.

An extension would assign meaningful weights w_{uv} to edges and optimize

$$\sum_{(u,v) \in E, p(u) \neq p(v)} w_{uv}.$$

Weights could be generated from:

- data dependency strength;
- number of bytes transferred between tasks;
- historical communication frequency;
- network latency between worker groups.

This extension would make the partitioning objective more faithful to real distributed systems.

5 Possible Extension 2: Improved Preprocessing

Graph preprocessing can strongly influence partition quality and runtime. A simple preprocessing extension is to remove isolated vertices, merge duplicate or nearly duplicate nodes, normalize edge weights, and order vertices by degree or locality before partitioning.

For the greedy baseline, better ordering can improve quality. The current implementation sorts vertices by weighted degree. More advanced orderings could use breadth-first traversal, graph degeneracy ordering, or spatial coordinates when available. For spectral methods, preprocessing could ensure that disconnected components are handled carefully, since Laplacian eigenvectors can be ambiguous when the graph has multiple components.

6 Possible Extension 3: Sparse Spectral Partitioning

The current spectral implementation uses dense eigen-decomposition, which is simple but not scalable. A stronger version would use sparse matrices and iterative eigensolvers to compute only the Fiedler vector. This would reduce memory use and make the method applicable to larger sparse graphs.

The algorithmic change would be:

1. build the Laplacian as a sparse matrix;
2. use an iterative solver for the smallest nonzero eigenpairs;
3. recursively bisect using the computed Fiedler vector;
4. fall back to simple splitting for tiny or disconnected components.

This extension directly addresses scalability and would make the spectral baseline more competitive for large graph instances.

7 Possible Extension 4: Multilevel Partitioning

A more ambitious extension is a METIS-style multilevel algorithm. Such an algorithm has three phases:

1. **Coarsening**: repeatedly match and contract vertices to create smaller graphs;
2. **Initial partitioning**: partition the smallest graph;
3. **Uncoarsening and refinement**: project the partition back to larger graphs and refine it at each level.

This would combine global scalability with local improvement. Kernighan–Lin or a related local-search method could be used during the refinement phase. This extension is natural because it connects the reproduced algorithm to one of the most successful practical graph partitioning frameworks.

8 Possible Extension 5: Stronger Distributed Optimization Simulation

The current consensus simulation counts cross-worker messages but does not make cross-worker communication slower or less reliable. A stronger extension would introduce a cost model:

- cross-worker messages have higher latency;
- cross-worker communication is allowed only under a budget;
- delayed messages affect the update rule;
- different partitions lead to different wall-clock convergence times.

For example, a delayed consensus update could distinguish local messages from remote messages. Local messages update immediately, while remote messages are applied after a delay. Then partitions with fewer remote edges may not only reduce communication count but also improve convergence per unit time.

Another extension is distributed gradient descent. Each vertex could represent a local objective $f_i(x)$, and agents could perform local gradient steps plus neighbor averaging. The partition would then determine which synchronization events are local and which are remote.

9 Project-Level Takeaway

The optional extensions strengthen the link between classical algorithms and modern applications. The implemented consensus simulation already shows that lower graph cuts translate into fewer cross-worker messages. The proposed extensions would make the system model more realistic and the algorithms more scalable.

The most valuable next step would be a weighted communication model combined with sparse spectral partitioning. This would preserve the current project structure while addressing two important limitations: uniform edge costs and dense eigen-decomposition. A larger future version could then add multilevel coarsening and compare directly with METIS-style tools.